Might I Get Pwned: A Second Generation Password Breach Alerting Service

***Abstract*—Credential stuffing attacks use stolen passwords to log into victim accounts. To defend against these attacks, recently deployed breach alerting services are providing APIs that help users and companies check whether a username, password pair is exposed. These services however only check if the exact password is leaked, and therefore do not mitigate credential tweaking attacks in which the adversary tries to compromise a user account with variants of their leaked passwords.**

We initiate work on password breach alerting APIs that protect users from credential tweaking attacks. The core underlying challenge is how to warn users about their password being similar to one in a breach, while preserving honest client's privacy and also preventing malicious clients from extracting breach data from the service. We formalize the problem, and explore a variety of ways to measure password similarity that balance efficacy, performance, and security. Based on this analysis, we design Might I Get Pwned, a new kind of breach alerting service that simultaneously helps protect users from state-of-the-art credential tweaking attacks while preserving user privacy and limiting potential exposure of sensitive breach entries. Finally, we show empirically that the proposed protocol is fast enough to be deployed in practice.

I. Introduction

The wide availability of password databases exposed due to breaches has made credential stuffing attacks the most widespread method for account compromise [17]. Here an adversary uses exposed passwords to try to log into user accounts on various web services, which succeeds frequently because of password reuse. In response, practitioners have setup third-party services such as HIBP [2, 20] and GPC [24, 26] that provide APIs to check if a user's password has been exposed in known breaches. These breach-alerting services, also called compromised credential checking (C3) services, can help prevent credential stuffing attacks by alerting users to change their passwords.

Existing C3 services, however, can only prevent credential stuffing attacks, and can leave users vulnerable to credential tweaking attacks [15, 22, 28]. Because users often select similar passwords (e.g., adding a '1' to the end of their old password) [15, 23, 29], attackers can submit guessed variations (tweaks) of a user's leaked password. Pal et al. [22] estimate via simulation that it is possible to compromise 16% of user accounts in less than a thousand guesses, should the attacker have access to a breached password of the user — despite the account being protected by a credential stuffing counter measure via an existing C3 service.

We therefore design a second-generation C3 service with the ability to notify users that have a password similar to a breached one. We call our design "Might I Get Pwned" (MIGP), as a tribute to the first-ever C3 service HaveIBeenPwned (HIBP) [27]. Users and web services can use MIGP to learn if an account could be vulnerable to credential tweaking attack, and take action accordingly.

Building a MIGP-type system required overcoming a number of challenges. Existing exact equality C3 services [20, 24, 26] scale to the billions of username, password pairs in known breaches [20] by splitting the dataset into smaller buckets and then running a cryptographic private set membership (PSM) protocol over the bucket relevant to a client's username, password pair (u, w). Buckets are defined by grouping all passwords that have matching hash-prefix of the username. Note that shorter prefixes are better for privacy, since the prefix, and therefore partial information about the username, is revealed to the server. We explore two potential types of approaches to additionally warn users should they query on (u, w') for w' similar to w.

The first is to replace the PSM step with a private similarity protocol (PSP). To measure similarity, one can utilize password embeddings [22]. These map a password to a vector over the real numbers in such a way that similar passwords are associated with vectors with the larger dot product. We use breach-data-driven simulations to show that one can set thresholds to warn users about most password variants that are vulnerable to state-of-the-art credential tweaking attacks [22], while retaining a relatively low false-positive rate (warning users about variants that are unlikely to be guessed by an attacker). We prototyped this approach using Facebook's open-source 2PC library, but for reasonable bucket sizes unfortunately is not yet fast enough for deployment. Whether other 2PC techniques can be made sufficiently fast remains an open question.

Our second approach can be viewed as simple brute-force: add to each bucket up to n password variants for each (u, w) pair in the bucket, and then perform the PSM over buckets as before. Performance degrades by a factor of n. We can extend the PSM to additionally reveal to the client whether an entry is an original breach entry or a variant, using a particularly efficient version of the metadata encryption technique described in Thomas et al. [26]. A priori, however, it is unclear if this brute-force approach can be rendered effective: it requires exploring the trade-off between warning users about sufficiently many variants to prevent attacks (where we would want ideally relatively large n) with storage and bandwidth complexity (where we would want ideally small n). Moreover, we need to understand the security risks associated with allowing potentially malicious clients to leverage the service to discover user's breached credentials, which may be made easier as n grows.

We use empiricism to explore different ways of enumerating

variants, and show via simulation how these fair against state-of-the-art credential tweaking attacks. We start with the deep learning approaches pioneered in prior work for generating variants, in particular the pass2path algorithm [22]. We show via simulation that for moderate values such as $n = 100$, a webservice deploying a pass2path approach would block 69% of attacks, despite the attacker having better knowledge of the password distribution and up to 1,000 guesses. One downside of pass2path is that it is computationally expensive to run, and so we also show how to match this performance using a more efficient weighted edit distance approach that empirically derives simple-to-apply mangling rules. This works because of the values of n of interest: the top $n = 100$ variants are relatively speaking simple to learn. For larger n pass2path outperforms the rule-based approach.

Finally, we explore the tension between n and abuse resistance. Malicious clients can try to abuse existing exact-equality C3 services to learn another user's breached password by simply guessing a sequence of likely passwords and interacting with the service once per guess. Slow hashing [1] and rate limiting make this difficult. MIGP-style services may make it easier for attackers, because they now need only to guess a variant of a target password. We formalize this threat model, and explore different adversarial client strategies. We show that finding an optimal sequence of guesses is NP-hard, and detail heuristic algorithms and evaluate them empirically for various values of n. Ultimately, we show that adding similarity checks improves adversarial success by up to 1.3 times within a reasonable guessing budget. We propose a simple-to-implement countermeasure that we show reduces this speed-up drastically: do not allow passwords or variants that fall on a blocklist consisting of the most popular passwords. Users should be warned to avoid such easy-to-guess passwords whether or not they appear in a breach.

We implement a prototype of MIGP, and show that it is practical to deploy. API query latency is dominated by the slow hashing and time to download the larger bucket but remains practical at less than one seconds on fast network connections. We conclude that the operational overhead of MIGP is reasonable compared to that of deployed C3 services, while providing better user security against state-of-the-art attacks.

Contributions In summary, the main contributions of this paper are the following:

- We introduce a new type of credential checking service, called MIGP, that helps mitigate credential tweaking attacks by letting users and web services learn if a user is using a password that is vulnerable to such attacks.
- We evaluate possible similarity measures and show that storing variations of a password is the most computationally efficient, while still being effective in thwarting credential tweaking attacks.
- We analytically and empirically analyze the threat of abusive clients attempting to exploit MIGP to learn breached credentials, and show that blocklisting popular passwords effectively reduces this threat.
- We prototyped the implementation of MIGP, and showcase its practicality to be deployed on a large-scale.

We have had initial discussions with major service providers about deploying MIGP, and to facilitate such deployments we will be making our prototype public and open source.

II. Background and Prior Work

Credential stuffing attacks and C3 services. Billions of passwords are available online as a result of compromises [17, 27]. Users often choose the same password for different web services, making credential stuffing attacks effective. C3 services have been adopted in client browsers or login server backends to check user credentials as a prevention measure. Existing C3 services include Have I Been Pwned (HIBP) [27], Google Password Checkup [24], and Enzoic [4]. HIBP [2] has publicly documented APIs to check if a username or password is in a breach. Several password managers such as 1Password and LastPass and browsers such as Firefox are using HIBP to warn users about their leaked passwords. This may result in false positives since common passwords will always be flagged.

Google Password Checkup (GPC) was released as a chrome-extension in 2019 [24] and later integrated into Chrome. The GPC protocol, which is described in [26], checks if a username-password pair is present in the leak, leading to less false positives compared to HIBP. Li et al. [20] formalized the security requirements of such systems in an honest-but-curious server setting, and analyzed via simulation the risk of GPC's initial algorithm which leaked some information about passwords to the service.

The state of the art C3 protocol proposed in [20, 26] now deployed by GPC handles the large scale of breach datasets using bucketization. To check a username, password pair (u, w), the client sends a bucket identifier b which is the first 16 bits of the cryptographic hash of u. In parallel, the client and server perform a private set membership (PSM) protocol to securely determine if (u, w) is in the bucket — the set of all (u', w') with the same username hash prefix. The PSM protocol is built using an oblivious PRF (OPRF).

Credential tweaking attacks. Currently deployed C3 services cannot warn users about a password unless the exact password is present in the breach. For example, a minor variation (such as adding "7" at the end) of the compromised password "yhTgi456" won't be detected by the C3 service. Users often pick similar passwords while resetting their passwords on a web service [29] or picking passwords for different web services [15]. These passwords are vulnerable to credential tweaking attacks [15, 22, 28], where the attacker tries different variations of the leaked password. Wang et al. [28] and Das et al. [15] used human-curated rules to generate guesses for a credential tweaking attack. Subsequently, Pal et al. took a data-driven, machine-learning approach to build similarity models

for passwords from the same user. They trained a sequence-to-sequence [25] style neural network model (pass2path) that outputs similar passwords given an input password. This is now the best known attack, with simulation showing that a pass2path-based attack can compromise 16% of accounts that appeared in a breach using at most 1,000 guesses, despite the use of a C3 service as a credential stuffing countermeasure. Pal et al. also showed in a case study that over a thousand accounts at Cornell University were at the time vulnerable to credential tweaking attacks, showcasing their practical risk.

Pal et al. also proposed a potential defense: a personalized password strength meter (PPSM) which considers the strength of a selected password based on its similarity to the user's leaked password(s). Their PPSM uses a password embedding, which maps a password to a d-dimensional vector in $\mathbb{R}^d$. Similarity between two passwords are defined as the dot product of their embedding vectors. Their PPSM can be integrated, for example, as part of a password-change workflow where it can detect if the new password is similar to the old one. But in general this won't help alert users that they are using a password that is too similar to a leaked one.

The question left open by prior work is whether we can build C3-like services that warn about vulnerability to credential tweaking attacks.

III. Overview of MIGP

In this paper we propose a new C3 service, called MIGP. It adds new features over prior approaches and, in particular, tackles the open question of how to warn users about passwords that may be vulnerable to credential tweaking attacks. In this section, we present a high-level overview of MIGP from the perspectives of functionality, security, and performance.

Service architecture and functionality. MIGP consists of a server storing a breach dataset $\tilde{S}$. Here $\tilde{S}$ is a set containing $|\tilde{S}|$ username and password pairs $\{(u_1, w_1), \ldots, (u_{|\tilde{S}|}, w_{|\tilde{S}|})\}$ where each $u_i \in U$ is a username and each $w_i \in W$ is a password. The sets U and W consist of all possible usernames and passwords. The functionality that MIGP targets is enabling a client to query to learn if the username-password pair $s = (u, w)$ is equal or similar to any leaked password in $\tilde{S}$ corresponding to the username u. The MIGP service, therefore, returns match if there is an exact match corresponding to s. In case there exists a similar password in the database, the service returns the value similar. We represent the set of passwords considered similar to w as the set $\tau(w)$. We will explore various ways to define τ. Finally, should $w' \neq w$ and $w' \notin \tau(w)$ then MIGP returns none.

Threat model. In this work, we explore two primary threat models: (1) an honest-but-curious server trying to learn information regarding a user's queried password, and (2) a malicious client querying the MIGP server to retrieve leaked passwords corresponding to other users. We will explain each in turn.

The MIGP server is assumed to be honest-but-curious: it doesn't deviate from the protocol but observes the protocol in an attempt to learn more information from the user queries. Ideally, we would like the MIGP server (or any C3 server) to learn nothing about the queried usernames or passwords. However, building a practical solution that achieves this requirement is hard given the huge scale of $\tilde{S}$ with billions of credentials. As mentioned in the last section, the state of the art protocols existing C3 services reveal some bits of information about the username to allow partitioning $\tilde{S}$ into smaller buckets on which a PSM protocol can be efficiently executed. Looking ahead, MIGP will extend this approach to perform a private similarity protocol (PSP) over the bucket. Ultimately we will use a straightforward approach for PSP which at core simply adds to $\tilde{S}$ a number n of variants of all the username, password pairs. The challenge lies in showing that one can find an algorithm for generating a limited number n of variants that sufficiently cover vulnerable passwords while not degrading performance prohibitively.

Clients using the MIGP service can also be malicious. In particular, they might mount a guessing attack in an attempt to extract username, password pairs from $\tilde{S}$. Learning a user's leaked password can help them compromise that user's accounts on other web services through credential stuffing and tweaking attacks. Prior work did not empirically analyze this threat for exact-check C3 services, but they did include anti-abuse countermeasures such as requiring computationally intensive slow hashing to complete a query [26].

For MIGP, however, this threat is more subtle, because now a malicious client may be able to speed up searches due to the service revealing when a similar password has been queried. We will analyze this risk analytically and empirically, and ultimately propose a simple countermeasure that mitigates it: blocklisting from inclusion in breach datasets the top (say) 1,000 most popular passwords and their variants. Users should be prevented from picking these easy-to-guess passwords anyway, regardless of breach status.

Performance requirement. Former C3 services like GPC have breached dataset $\tilde{S}$ containing more than 4 billion leaked credentials which is going to increase in coming years with more credential breaches. Cloudflare, which helps host HIBP, reports processing around 600,000 requests per day on average [3]. Thus, MIGP will need to scale to billions of entries and millions of requests per day. Ideally we will avoid computationally expensive tasks for the server when replying to queries. We will also want fast query responses and bandwidth utilization, ideally similar or better than existing C3 services like GPC and HIBP.

The rest of the paper is organized in the following way. Next we will turn to understanding similarity measures. Then we will turn to analyzing the efficacy of malicious clients attempting to extract credentials from a service that uses different password similarity models, and showcase how blocklisting can be helpful in mitigating this risk. All this will educate our final design for MIGP, which we give in Section VI, along with an experimental performance evaluation of our prototype implementation. Finally, we end with a discussion

of deployment considerations in Section VII.

IV. Effectiveness of Different Similarity Measures

In this section, we define different ways of determining whether a user's password $\tilde{w}$ is similar to a leaked password w. There are several potential similarity measures we could use, such as the one proposed in [22] or simple weighted edit distance. We will compare different similarity measure based on their *efficacy* at protecting from credential tweaking attacks, *performance* of computing the similarity measure in our MIGP setting, and *security* against malicious clients attempting to extract passwords from a MIGP server. We focus on the first two in this section, and discuss the latter in Section V.

A. Private similarity protocols

The MIGP system performs a private similarity protocol (PSP) to check for similar passwords. We explore two techniques to implement effective and efficient PSP. The first technique builds off a private set membership (PSM) protocol. Given a password w we can use a generative language model to compute a set $\tau_n(w)$ of n similar passwords. Then we can check if $w' \in \tau_n(w)$ in a private manner using a PSM protocol. In fact, we will eventually be checking username, password pairs (see Section VI), but for simplicity in this section, we focus on just comparing passwords.

The second method takes advantage of an embedding model $v : w \rightarrow \mathbb{R}^d$ that maps passwords to d-dimensional vectors. The embedding model v is learned from breach data (c.f., [22]) in order to have vectors of similar passwords be closer in the vector space. In more detail, we set a similarity threshold θ and deem two passwords w and w' to be similar should $v(w) \cdot v(w') \geq \theta$, where '$\cdot$' denote the dot product between the vectors. We can use a secure 2PC protocol to perform the private dot product and comparison. Looking ahead, we will end up taking the first approach rather than this one because of performance issues.

B. Similarity checking algorithms

We primarily examine five candidate techniques for computing similarity between passwords. Four out of these five techniques provide generative methods, that is to say they take as input a password and output an ordered list of similar passwords. An observation is that any credential tweaking attack can be repurposed to be a generative method for similarity. We therefore start with the attack algorithms of Das et al. [15] and Pal et al. [22]. We denote these by Das and P2P, respectively. We also created more efficient and effective variants of these methods — Das-R and wEdit — as we show below. The fifth similarity technique we explore uses the password embedding model proposed by [22].

Das-R. We observed that ordering of the tweaks used in Das is not effective for smaller n. So we reorder the set of tweaks based on the frequency with which these tweaks explain the difference between passwords associated to the same users in the MIGP dataset. (The details of the dataset are discussed in Section IV-C.) As we show the reordering significantly improves the efficacy of the rules for smaller values of n. We call this similarity measure Das-R. The top ten similarity rules for this strategy are given in Appendix VIII-A; examples include swapping the case of the first character if it is a letter, etc. Not all rules apply to all passwords, in which case we continue applying further mangling rules until we obtain n variants.

wEdit. While the Das and Das-R approaches use hand-derived rules, we also explore automatically deriving a ranked list of mangling rules that can be applied to a password to obtain variants. Of course mangling rules have long been used in password cracking systems (e.g., [5]), but here the goal is different as we want to derive rules that work well to identify, given an input leaked password, those similar passwords that are likely to be vulnerable to credential tweaking attacks.

In more detail, a *unit transformation* specifies a specific edit to apply to the original password. A unit transformation is defined by a tuple (e, c, l) where e specifies the edit type as one of insertion, deletion, or substitution; c denotes the character to be inserted or substituted ($c = \perp$ for deletion); and l is the location for the edit. The location numbering is length-invariant, representing the distance from the first character by positive numbers and last character by negative numbers. Every position therefore has two potential locations; we use the smaller of the two distances and break ties using a positive distance (from the start of a password). For example, $(\mathsf{insertion}, 0, -1)$ specifies adding the character '0' to the end of a password while $(\mathsf{substitution}, a, 2)$ specifies replacing the second character with a lower case letter 'a'.

Given a pair of passwords (w, w'), we can calculate the shortest sequence of unit transformations to generate w' from w — we refer to this as the *transformation path*. The computation can be done using standard edit distance algorithms. The length of a transformation path is the number of unit transformations it contains. In fact, we use the keypress representation of the passwords w, w' as defined in [12], which includes special characters such as as shift and caps lock.

Given a breach dataset that contains multiple passwords associated to the same user accounts, we compute a ranked list of transformation paths as follows. For every (w, w') pair associated to the same user account, we compute the transformation path to convert w to w', stopping if it gets beyond length three. (We could consider longer paths, but found that three produced sufficiently good results.) We then order the resulting paths by their frequency — how many password pairs each path explains.

To generate variants of a password w, we attempt to apply the transformation paths one at a time, in decreasing frequency order. Should a transformation path not be applicable to w we skip it and go on to the next path, stopping when we have generated n variants.

Note that wEdit contains a much more exhaustive list of mangling rules (transformation paths) compared to Das-R. However, wEdit is not sensitive to the input and the hand-

	D	S	T	S_1	S_2
# users	908	760	230	380	380
# passwords	438	373	119	210	210
# user-pw pairs	1,317	1,069	248	535	535

Fig. 1. Number of entries (in millions) in the entire dataset T, breach dataset S, and test dataset T. S_1 and S_2 are equal splits of S on user-pw pairs.

crafted rules in Das-R includes ones like insert '3' if the last character of the word is '2'. Nevertheless we'll see below that they have similar efficacy in measuring similarity.

In the coming paragraphs, we will compare empirically different similarity measures for MIGP from the perspective of efficiency and efficacy.

C. Datasets and Simulation Approach

To drive empirical evaluation of the five similarity approaches, we turn to the dataset used in [20]. Their dataset is a compilation of billions of breached credentials from various websites. The compiled leak is publicly available on the internet and contains 1.4 billion username-password pairs.

We cleaned the dataset and removed non-ASCII characters and passwords longer than 30 characters. We also removed passwords corresponding to username having more than 1000 passwords as it may refer to default account names used on multiple sites (such as 'admin'). We merged usernames based on the mixed-method from [22]. The resulting dataset D consists of 1.3 billion unique username-password pairs (Fig. 1).

We partitioned D into two for simulation purposes. The major split (80%) simulates the publicly leaked dataset, S. We further divide the breached dataset S into two equal non-intersecting sets S_1 and S_2, for reasons that will be made clear below. The remaining dataset (20%) is the testing dataset, T, that will be used to simulate users' actual credentials that an attacker wants to infer. The test dataset can, therefore, be considered a list of users' current password on some target website for which the attacker wants to gain illicit account access. The testing dataset is neither accessible to the attacker nor to the similarity mechanisms that we train.

In Fig. 1, we report some of the statistics on the dataset splits. S and T consist of 760 and 230 million unique usernames respectively. About 82 million usernames are present in the intersection of T and S, implying that these users in the test dataset have at least one password in the simulated breach dataset. The number of users, passwords, and user-password pairs are similar for S_1 and S_2.

The attack simulations are conservative in that the attacker always has access to more data than is used to train the similarity model. That is, we provide the attacker with the entire leaked dataset S but train the similarity mechanism only on S_1; half of the leaked dataset S.

D. Defense accuracy

We examine the effectiveness of using different similarity measures in the MIGP system. We investigate the similarity measures based on security guarantees and usability. There exists friction between providing strong security guarantees against credential tweaking attacks and making sure that the user is not given false or numerous warnings to change their password, which can lead to user fatigue.

Parameters	Similarity measures	True positive	False positive
$n = 10$	Das	33.2	0.6
	P2P	46.4	0.0
	Das-R	**52.6**	0.0
	wEdit	49.6	0.0
$n = 100$	Das	46.9	2.2
	P2P	**69.0**	0.1
	Das-R	63.5	0.2
	wEdit	**69.3**	0.1
θ = 0.83	PPSM	**67.9**	2.0
θ = 0.75		87.6	4.7
θ = 0.5		99.1	14.0

Fig. 2. True positive and false positive rate for different similarity measures. All the values are in percentage of total samples (%).

We sample password pairs for our experiment from two categories: (1) passwords vulnerable to a credential tweaking attack, which MIGP should classify as similar (used to measure true positives), and (2) passwords belonging to the same user but which are not vulnerable to the credential tweaking attack and therefore MIGP should not classify as similar (used to measure false positives). The idea is to measure the performance of different similarity measures against the different classes of password pairs. Each password pair (w_1, w_2) consists of a password (w_1) selected from S_1 and another (w_2) from T. Hence, both the attacker and MIGP knows the breached password w_1 corresponding to the target user and want to attack/protect the unknown (test) password w_2 of the user.

For the first category, we consider that w_1 and w_2 belong to the same user and are vulnerable to the best-known credential tweaking attack pass2path [22]. We assume the attacker has access to the leaked dataset S (therefore including w_1) and tries to compromise w_2. The attacker trains their attack model using S and mounts the credential tweaking attack. We sample 10,000 password pairs that were guessed by the attacker in less than 1000 guesses and label them as vulnerable.

For the second category of false positives, we consider w_1 and w_2 belonging to the same user but w_2 is not guessable by a credential tweaking attack given w_1 (in 1,000 guesses). The idea is that flagging these passwords as similar would represent false positives, which unnecessarily burden users with spurious warnings.

Result. Fig. 2 shows the performance of the similarity measures against vulnerable pairs of passwords. As expected, increasing the number n of similar passwords improves the coverage against vulnerable-pairs across all methods. However, the false-positive rate also increases with larger n.

Das-R give the maximum 52.6% coverage for $n = 10$ tweaks among all the generative approaches. P2P and wEdit perform the best for $n = 100$ with around 69% accuracy. PPSM gives high coverage against the attacks but also has a higher false-positive rate compared to the generative ap-

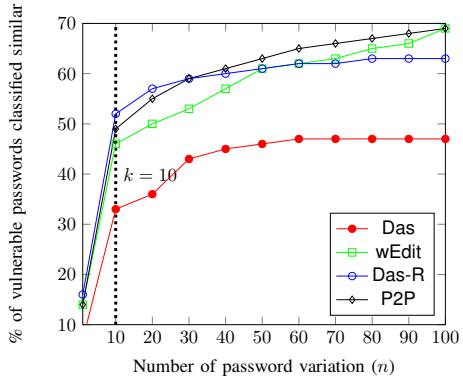

Fig. 3. Percentage of pass2path vulnerable password pairs marked similar for different number rule-based similarity measure. The slope of the graph decreases with the increase in k.

proaches. We select the threshold of PPSM to be 0.83, as the false positive rate is around 2%, which is similar to the maximum among generative algorithms for $n = 100$.

Fig. 3 examines the performance of the four generative models as a function of the number of tweaks n considered. Although increasing n increases the attack coverage, the slope of the graph decreases rapidly. Therefore, the benefit of considering a higher n value diminishes with increasing bandwidth and storage cost. The slope is maximum at $n = 10$.

With the increase in n, Das-R outperforms other techniques up to $n = 30$. P2P outperforms the other measures after that. However, wEdit catches up near $n = 100$. It was surprising to us that the rule-based approaches (Das-R, wEdit) end up matching or exceeding the performance of the much more complex deep learning approach underlying P2P. The reason seems to be that for low values of n, rule-based approaches can easily capture the "head" of the distribution of very frequently seen variants. As one goes deeper into the distribution, the deep learning approach works better by at finding and ordering less frequently seen similarity relationships. For example, P2P outperforms wEdit for $n = 1000$ by 4%.

E. Assessing Performance Feasibility

A key deciding factor in which similarity measure to use in practice with MIGP is their performance. In this section, we compare the performance of different similarity rules and provide estimates. We select the top four similarity measures, therefore excluding Das. Our goal is to understand the feasibility of their deployment as PSP protocols; we evaluate the full efficiency of our selected approach operating within MIGP later in the paper. The three generating algorithms — P2P, Das-R, and wEdit — perform a PSM with the breached and similar passwords. PPSM finds similarity by mapping the passwords to vectors and comparing their dot product to a threshold. Therefore, the implementation of PPSM based MIGP requires the private dot product computation followed by a private comparison.

To estimate feasibility, we simply use a bucket of size 10,000 username, password pairs. We measure the cost of the techniques to perform similarity matching on this stored bucket. We use $n = 100$ for P2P, Das-R, and wEdit, while we set the PPSM threshold to 0.83. Timings were performed on a desktop with an Intel Core i9 processor and 128 GB RAM, and here we run the entire protocol within the same machine. P2P uses an Nvidia GTX 1080 GPU along with the processor to run the pass2path neural network.

Similarity measure	Latency	B/w	Compat.	Storage per bucket	Precomp.
wEdit ($k = 100$)	< 1 sec	14 MB	Yes	14 MB	41 sec
P2P ($k = 100$)	< 1 sec	14 MB	Yes	14 MB	180 sec
Das-R ($k = 100$)	< 1 sec	14 MB	Yes	14 MB	0.5 sec
PPSM ($\theta = 0.83$)	120 sec	80 KB	No	8 MB	1 sec

Fig. 4. Performance summary of the most effective similarity measures. The techniques are compared based on different tradeoffs. All the numbers are based on a bucket of size 10,000. The trade-off are also ranked left to right based on the importance to deployment. Here latency does not included n/w or i/o cost.

We summarize the results in Fig. 4. We explain the other columns below.

Latency. An important performance metric for real-world deployment is query latency. We measure the latency to compute the cryptographic protocol on the bucket. The OPRF based PSM protocol (P2P, Das-R and wEdit) is implemented in Python and uses petlib[1] library's secp256k1 elliptic curve implementation. We implement the PPSM based protocol using Facebook's Private AI library Crypten.[2] Both P2P and wEdit take less than 1 second, compared to a full two minutes for the PPSM approach. Note that these measurements, which do not include network latency, should be considered lower bounds on performance. We give more detailed analysis of performance of MIGP later in Section VI-B.

Bandwidth. We estimated the bandwidth sent from the server to the client. The generative algorithms hashes each of the username-password pair to 128 bit (or 16 byte) strings (we discuss the details of the protocol in Section VI), leading to 14 MB. PPSM sends back a long tensor of size 10^4 — one for each username-password pair, which is total 80 KB.

Compatibility. The compatibility column ("Compat") indicates whether the service can be built on top of existing cryptographic protocols used for deployed exact equality C3 services, which would ease deployment.

Storage. Crypten encrypts the 100 dimension password embedding vector to generate a 100 dimension long tensor. Therefore the bucket takes 8 MB, compared to 14MB for the other three techniques.

Precomputation. The precomputation time to generate wEdit and Das-R guesses took 41 and 0.5 second respectively without any parallelization. P2P takes 3 min in the GPU with a batch size of 100. PPSM takes 1 sec to generate the vectors.

[1]https://github.com/gdanezis/petlib
[2]https://github.com/facebookresearch/CrypTen

The columns of Fig. 4 are ordered from left to right in decreasing order of our perception of how critical this aspect of the protocols are to deployment. wEdit, Das-R, and P2P take less computation time compared to PPSM and have the beneficial feature of being mostly compatible with earlier C3 services. PPSM also requires more network round trips to perform the 2PC computation. We therefore focus on the generative methods. We leave as an open question whether one can make another 2PC-based protocol fast enough for reasonably sized buckets.

Among the generative methods, at Das-R outperforms the other protocols for $n = 10$. It is also simple and easy to deploy. For $n = 100$, wEdit and P2P have better efficacy, but among the two wEdit is drastically simpler to deploy and faster to run, requiring less pre-computation time and no GPUs. Therefore in the rest of the paper, we focus on Das-R for $n = 10$ and wEdit for $n = 100$.

V. Breach Extraction Attacks against Similarity Checking Services

Similarity protocols, including MIGP, allow clients to check whether a password similar to the queried one is present in the breach. Current C3 services, such as GPC [26], do not reveal any information about the breach data unless a client queries the exact username-password pair. A natural question is: Will a move to similarity protocols degrade the confidentiality of the username-password pairs in the leaked database?

Towards answering this question, we formalize the abstract setting of a malicious client that has access to a similarity oracle. The latter represents the similarity protocol, assuming it is cryptographically secure, and we refer to this as the ideal functionality (following parlance from the 2PC literature). We show that finding optimal guessing strategies is NP-hard, and then give heuristics that we evaluate empirically.

A. Security Games and Computational Hardness

Let $\mathcal{W}$ be a set of all possible strings up to certain length (say, 30), and $W \subseteq \mathcal{W}$ be the set containing all possible password strings for users U. We also assign a probability distribution p to all the passwords W representing the probability of a user $u \in U$ selecting the password $w \in W$. We define $\tau : W \to 2^{\mathcal{W}}$ such that $\tau(w)$ is the set of passwords defined to be similar to w, based on the similarity methods defined in the previous section. Therefore, if $\tilde{w} \in \tau(w)$, for some leaked password w for a user u, then MIGP will respond with "similar" when queried with (u, w). In Section IV we show how to instantiate the function τ. We will assume $w \notin \tau(w)$, and $n = \max_w |\tau(w)|$.

The security games for computing the advantage of an attacker given the ideal functionality is given in Fig. 5. In the game MIGPGuess, the adversary is given an oracles access to MIGP which provides the ideal functionality. Note that the oracle is parameterized by a leaked password w chosen by the game and a similarity measure τ.

The goal of the attacker is to find the target password w in the minimum number of guesses to the oracles. The advantage

$\mathsf{MIGP}(w')$	$\mathsf{MIGPGuess}(\mathcal{A})$:
$q \leftarrow q + 1$	$w \leftarrow_p W$
if $w' = w$ then	$q \leftarrow 0$
return match	$w' \leftarrow \mathcal{A}^{\mathsf{MIGP}}$
if $w' \in \tau(w)$	if $w = w$
return similar	return q
else return none	else return $\lvert W \rvert$

Fig. 5. The functional abstraction of existing C3 services (IDBP) and MIGP (MIGP). The services are matching against the stored leaked password w. On the right we show different security games with attacker having access to IDBP and MIGP oracles.

of an attacker is measured as the expected number of queries to the respective oracles. (We assume the attacker has an unlimited query budget to the oracles.) This is also known[3] as α-guesswork (G_α) with $\alpha = 1$ [8]. An alternate approach to compute attack success is using q-success rate (λ_q), which measures the probability of guessing a password given q guesses. As shown in [8], these two notions are equivalent in the sense that the guessing strategy that minimizes G_α will also maximize λ_q (if $\alpha \geq \lambda_q$).

MIGPGuess game. The MIGPGuess adversary tries to guess the exact target password w given access to the $\mathsf{MIGP}(\cdot)$ oracle. The advantage of the MIGPGuess adversary $\mathcal{A}$ is defined as the expected guess rank:

$$G^{\mathsf{MIGP}}(\mathcal{A}) = \mathbb{E}\left[\,\mathsf{MIGPGuess}(\mathcal{A})\,\right].$$

An MIGPGuess-adversary $\mathcal{A}^*$ is optimal if for all MIGPGuess-adversaries $\mathcal{A}$ it holds that $G^{\mathsf{MIGP}}(\mathcal{A}^*) \leq G^{\mathsf{MIGP}}(\mathcal{A})$. The optimal adversary $\mathcal{A}^*$ builds a ternary decision tree to query MIGP such that the expected guess rank is minimized. As we show below building such decision tree that minimizes the guesswork is NP-hard, therefore the optimal attack against MIGP is also NP-hard.

Definition 1 (Optimal MIGP guess problem)**.** *Given (W, p, τ), we define optimal MIGP guess (OMIGP) problem as building the query tree for $\mathcal{A}^*$ that minimizes the guesswork for distribution p over W with MIGP similarity measure being τ.*

Theorem V.1. *OMIGP problem is NP-hard.*

To prove this result, one might be tempted to reuse a result from Chatterjee et al. [11, 13], who investigated a similar setting in the context of typo-tolerant password checking. They formalized guessing attacks against a server that allows the user to login with a small set of typos. Although their setting is similar to ours, there is one crucial difference: MIGP reveals with whether the query is a match or is similar to a password, while the password typo correction oracle does not distinguish does not reveal whether the password was an exact or near match. This seemingly minor distinction means we can't use their techniques, and we had to find another proof strategy.

Therefore we show that we can reduce the optimal binary decision tree (OBDT) problem to OMIGP in polynomial time. Because, OBDT does not have an polynomial time

[3] α-guesswork measures the expected number of guesses to make to ensure the probability of guessing a randomly chosen password is at least α.

solution [19] then OMIGP also cannot have a polynomial time solution.

Binary decision tree (BDT). The instance of binary decision tree (BDT) is defined as a three tuple (X, p_X, Q), where X be the set of n items $x_1, x_2, \ldots, x_n$, p_X is a probability distribution defined over the elements of X, and Q is a set of m questions (functions) such that $Q_i : X \mapsto \{0, 1\}$. The goal is to find a decision tree that completely characterizes X using the questions in Q as decision nodes. The questions $q \in Q$ is specified in all the internal nodes while the items x_i fill the root nodes of the tree. The expected depth of the tree is defined as $\sum_{x \in X} p_X(x) \cdot d(x)$, where $d(x)$ is the depth — distance from the root — of the element x in the binary tree.

Definition 2 (Optimal MIGP guess problem)**.** *Given (X, p_X, Q), the problem of finding optimal BDT (OBDT) is to build a binary decision tree that has the least expected depth, $\sum_{x \in X} p_X(x) \cdot d(x)$.*

Theorem V.2 ([19])**.** *OBDT problem is NP-hard .*

This follows from the proof provided by the classic results of Laurent and Rivest [19].

Theorem V.3. *OMIGP problem is NP-hard.*

Proof: We show that OMIGP is NP-hard by giving a polynomial time reduction of an arbitrary instance of OBDT problem to OMIGP. That means if there exists a polynomial time solution to OMIGP, then we will be able to solve OBDT in polynomial time as well. As we know that OBDT NP-complete [19], then there cannot be any polynomial time solution to OMIGP, therefore there cannot be any polynomial time solution for OMIGP.

An instance of OMIGP problem is defined as (W, p, τ), where W are the set of strings, p is a probability distribution over W, and τ is a similarity measure. Given an instance of BDT problem (X, p_X, Q), we can construct an instance of OMIGP problem as follows. For this we set $W = X \cup Q$ (We assume there is a unique way to represent elements in Q to be distinct from X.); $p(w) = p_X(w)$ if $w \in X$, and 0 otherwise; and $\tau(w) = \{y \mid Q_w(y) = 1\}$ if $w \in Q$, and $\varnothing$ otherwise.

Claim 1. *If the optimal ternary decision tree to the OMIGP problem instance (W, p, B) is T, then we claim that we can build a optimal BDT for the problem instance (X, p_X, Q) in polynomial time.*

Let T be the optimal decision tree for OMIGP problem instance where each node has three children for each type of MIGP output. The leaf nodes of the tree are the passwords in W. The distance of a password w from the root is the number of queries it take to guess w, which we denote as $d(w)$ here. As T has the minimum guesswork possible, $\sum_{w \in W} p(w) \cdot d(w)$ is minimum. As $p(w) = 0$ if $w \in Q$ (as per the reduction above), T ensures $\sum_{w \in X} p(w) \cdot d(w)$. This is the same as the property of OBDT. We, therefore, build the required binary decision tree T' by removing the edges for the exact match of the questions ($w \in Q$).

```
GreedyOBDT(X, p_X, Q):
for i ∈ {0,1} and j ∈ {1,...,|Q|} do
    X_j^i ← {x ∈ X : Q_j(x) = i}
T ← ∅
if |X| = 1 then  return X
j* ← argmax_j p_X(X_j^0) · p_X(X_j^1)
T.root ← j*
T.left ←GreedyOBDT(X_{j*}^0, p_X, Q)
T.right←GreedyOBDT(X_{j*}^1, p_X, Q)
return T

GreedyOMIGP(W, p, τ):
T ← ∅
if |W| = 1 then  return W
j* ← argmax_j p(B(w_j))(1 − p(B(w_j)))
T.root ← j*
T.middle ← j*
T.left←GreedyOMIGP(B(w_{j*}), p, τ)
T.right←GreedyOMIGP(W \ B(w_{j*}), p, τ)
return T
```

Fig. 6. Greedy algorithm for optimal binary decision tree and optimal MIGP. Here $B : \mathcal{W} \mapsto 2^W$, is a function such that $B(\tilde{w}) = \{w \in W \mid \tilde{w} \in \tau(w)\}$.

Thus we show, we can reduce an instance of OBDT problem into an instance of OMIGP problem, and the solution of OMIGP will provide a solution to OBDT. This contradicts the fact that OBDT is NP hard, therefore, OMIGP cannot have a polynomial time solution. This concludes the proof. ▌

The BDT problem will have a unique solution only if $m \geq \log_2 n$ and no two objects have same output for all the questions. Since BDT reduces to OMIGP, the same conditions apply for OMIGP as well.

B. Greedy approximation of OBDT and OMIGP

Approximate Greedy Algorithm As shown in Section V-A, finding optimal guessing strategy that minimizes the expected guess rank is NP-hard. However, attackers could still find approximate solutions that minimizes the expected guess rank. We give one such greedy algorithm for OMIGP, which we call GreedyOMIGP, in Fig. 6 based on the greedy algorithm GreedyOBDT for OBDT (shown in Fig. 6 provided by Chakaravarthy et al. [10]). Chakaravarthy et al. also has show that the approximation factor for the greedy algorithm GreedyOBDT is $\mathcal{O}(\log |X|)$. In Appendix VIII-B, we show that the expected guesswork due to the greedy algorithm GreedyOMIGP is at most $\mathcal{O}(\log |W|)$ factor of the minimum expected guesswork $G^{\mathsf{MIGP}}(\mathcal{A}^*)$. This approximation factor is quite large, especially when $|W|$ is very large. Nevertheless, this shows that it is possible to compute approximate solutions that might help an attacker guess a user's leaked password stored in MIGP server more effectively (than existing C3 services). We will therefore empirically verify the efficacy of the greedy algorithms in Section V-C and show that the success rate can be quite large.

C. Empirical Security Evaluation

MIGP service could be abused by malicious clients to learn about the leaked passwords of a user. Although finding the

```
GreedyMIGPGuess(W, p, B, q):
W' ← ⋃_{w∈W} τ(w) ∪ {w}
for i ← 1 to q_c do
    w̃_i ← argmax_{w̃∈W'} p(B(w̃))
    r ← MIGP(w̃_i)
    if r = none then
        W ← W\B(w̃_i)
        for w ∈ B(w̃_i) do   p(w) ← 0
    else if r = similar then
        for w ∈ W\B(w̃) do   p(w) ← 0
        W ← B(w̃_i)
        W' ← ⋃_{w∈W} τ(w) ∪ {w}
    else if r = match then
        return w̃_i
    W' ← W' \ {w̃_i}
return arg max_{w∈W} p(w)
```

Fig. 7. Greedy algorithms for finding q guesses to MIGP oracle (which is parameterized with the target password w^* chosen randomly from p). Here $B(\cdot)$ is defined as $B(w') = \{w \in W \mid w' \in \tau(w)\}$ for any $w' \in W'$.

optimal strategy for doing the attack is NP-hard (Section V-A), it is possible to create efficient greedy approximate algorithms. We discuss approximations in detail in Appendix VIII-B, including bounds on approximation factors. Here we measure the efficacy of a greedy algorithm to understand the real-world threat of a malicious client attempting to extract data from the MIGP service. As above, we abstract out the details of the service and assume that the attacker has access to a MIGP oracle that takes a password as input and returns $\{\mathsf{match}, \mathsf{similar}, \mathsf{none}\}$.

Instead of computing guesswork — which might require making a large number of guesses to MIGP — we fix the query budget to measure the success rate of an attacker in guessing a target password. The games remain almost the same as in Fig. 5, except they return true or false, instead of the number of queries. This setup is also more realistic assuming the MIGP server will deploy some form of rate-limiting to queries from a client. As we show in this section, the rate-limiting is crucial for security and discuss potential privacy-preserving ways to rate limit in Section VII.

We define *ball* of a variation $\tilde{w} \in \mathcal{W}$ as $B(\tilde{w}) = \{w \in W \mid \tilde{w} \in \tau(w)\}$. The ball represents the set of passwords that share a common variation. The probability of a ball $p(B(\tilde{w}))$, also called the weight of a ball, is the sum of the probabilities of the passwords in the ball.[4]

The attacker begins with a set of potential passwords W of the target user. In iteration i, the attacker picks the guess $\tilde{w}_i$ that has the highest ball weight, and based on the response from the MIGP oracle, it updates the set of potential passwords. In particular, if the response is none, then it removes all the passwords in the ball of the guess $\tilde{w}_i$ from W. If the response is similar, then it knows that the target password is one of the password in the ball of the guess $\tilde{w}_i$. If it the response is match, then it stops and outputs the guess $\tilde{w}_i$ (and wins the game).

[4]Chatterjee et al. [11] used similar algorithm to identify the advantage of an attacker against an authentication server that attempts to auto-correct certain typos in the user-entered passwords.

The complexity of the algorithm is $\mathcal{O}(|W| \times q)$, assuming there is a constant time algorithm to find the ball of a string. Note the attacker can decide on W that it believes will likely contain the target password. This could be popular passwords from prior public password breaches.

Experiment setup. For simulation we assume, the MIGP oracle is instantiated with S_1 data, and the attacker has access to only S_2 portion of the data. This simulates the situation where the attacker does not know some leak data present in MIGP, and it is trying learn those leaked passwords for a user. We sample 25,000 username-password pairs from S_1. For each pair, the attacker is given the username and required to find the target password given $q \leq 10^3$ queries to MIGP oracle. We compute the efficacy of an attack as the fraction of username-password pairs that the attacker can successfully guess. (As per our data division, none of the target usernames are present in S_2, and therefore the attacker cannot attempt a credential tweaking attack.) We will evaluate the security loss for two different similarity measures (τ) that performed the best (in efficacy and performance, see Section IV): (a) We pick $n = 10$ variants based on Das-R rules, and (b) $n = 100$ variants based on wEdit rules.

The S_2 dataset has 210 million passwords. If the attacker sets W to all the password in S_2 it will make GreedyMIGPGuess very slow to run. We instead picked the top one million passwords as W for the attack. These passwords are used by 24.34% of users in S_2.

For comparison, we also simulate a C3 service that do not provide checking for similar passwords, which we refer to as MIGP service with $n = 0$. For this case, the attack is simpler: the attack queries MIGP service top-q passwords, and if any response is match, it outputs that queried password.

Blocklisting. As noted earlier the MIGP service can block a number of most popular passwords from being queried, by removing them from its database. Therefore, the attacker learns no information about the target password by querying the MIGP server with any blocklisted password. We will assume MIGP blocks top-β passwords according to S_1 and their k variations (according to the setup). The attacker is aware of the set of blocklisted passwords and their variations. We experiment with $\beta = \{0, 10, 10^2, 10^3\}$.

Results. The success probability of the attacker in guessing the target password in $q \in \{10, 100, 1000\}$ is shown in Fig. 8. An attacker can learn 6.15% of passwords in less than a thousand guesses from C3 services that do not support any similarity checks ($n = 0$). The attacker's success probability increases to 14.3% against a MIGP service with $n = 10^2$. Providing information about the similarity of passwords leads to 2.3x increase in an attacker's ability to recover breached passwords. This shows that one must take caution to provide MIGP service. The abuse prevention (via slow hash function) mechanism in current C3 systems is not effective in blocking these attacks.

We measure the effect of blocklisting top β passwords

n	β	$q = 10$	$q = 100$	$q = 1000$
0	0	1.56 ($\pm$ 0.01)	3.08 ($\pm$ 0.05)	6.15 ($\pm$ 0.01)
	10	**1.13** ($\pm$ 0.01)	2.34 ($\pm$ 0.03)	5.33 ($\pm$ 0.02)
	10^2	**0.80** ($\pm$ 0.00)	**1.36** ($\pm$ 0.01)	3.98 ($\pm$ 0.03)
	10^3	**0.72** ($\pm$ 0.00)	**0.90** ($\pm$ 0.00)	**2.34** ($\pm$ 0.00)
10	0	2.03 ($\pm$ 0.03)	4.21 ($\pm$ 0.05)	11.00 ($\pm$ 0.15)
	10	**1.18** ($\pm$ 0.01)	3.14 ($\pm$ 0.01)	9.87 ($\pm$ 0.11)
	10^2	**0.86** ($\pm$ 0.00)	**2.17** ($\pm$ 0.03)	7.98 ($\pm$ 0.09)
	10^3	**0.76** ($\pm$ 0.00)	**1.32** ($\pm$ 0.01)	**4.84** ($\pm$ 0.09)
10^2	0	1.86 ($\pm$ 0.02)	4.56 ($\pm$ 0.03)	14.33 ($\pm$ 0.10)
	10	**1.15** ($\pm$ 0.01)	3.29 ($\pm$ 0.01)	10.01 ($\pm$ 0.08)
	10^2	**0.85** ($\pm$ 0.00)	**2.32** ($\pm$ 0.01)	8.12 ($\pm$ 0.08)
	10^3	**0.73** ($\pm$ 0.00)	**1.37** ($\pm$ 0.01)	**5.76** ($\pm$ 0.07)

Fig. 8. Attack success rate given different query budgets (q) for different attack scenarios. $n = 0$ (first set of rows) emulate the situation of existing C3 services, where the client can only learn if the exact queried password is present in the leak or not; for $n = 10$, MIGP oracle uses Das-R similarity rules, and for $n = 10^2$ it uses wEdit similarity rules. The service blocks most frequent β passwords. All success rates are in percent (%) of the total number ($(25,000)$ of samples from S_2. Lower values imply better security. The standard deviations are measured across the 5 random folds of these pairs.

in MIGP such that the attacker learns no information about them from MIGP should a user's password be one of them. Blocklisting consistently reduces the attacker's success across all values of n. For $q \leq \beta$, the success probability of an attacker for $n = 10$ and $n = 10^2$ setting remains below that of $n = 0$ and $\beta = 0$. We highlight those numbers in the figure. This is as expected, as because top β passwords are blocklisted the attacker learns no information about them from by querying MIGP. Note however, attacker still learns about the password that are outside top β passwords, and therefore, it can still obtain significantly more success than guessing only the most probable password.

Large blocklist will increase false positive rates of the service. However, top 10^3 passwords are not safe to be used in any context, and therefore, flagging them (even if falsely) is acceptable.

Query budget q towards MIGP service is another parameter affecting the attacker's accuracy. Reducing the value of q leads to better security. However, current C3 systems don't have a proper rate-limiting mechanism. We show that decreasing q by a factor of 10 lowers the security loss by 50%, highlighting the need for an effective rate-limiting technique in current and future C3 services.

Another interesting pattern to note is that for $q = 10$, the security loss decreases with the increase of n from 10 to 100. This is because large n produces larger balls, and therefore, for a small query budget $q = 10$ guessing the most weighted password ball may not be the optimal strategy. Especially even after the attacker learns that the target password is in the ball, it takes lot more query to identify which password is it in the ball. Therefore, identifying a variation of the leaked password doesn't guarantee that attacker will correctly guess the target password.

Overall, this analysis shows that with blocklisting size $\beta = 10^3$ and effective rate limiting $q \leq 10^3$, it is possible to reduce the success probability of breach extraction attack to below what it is for traditional C3 services (with $n = 0$).

Pre-computation by MIGP Server
Server's secret key: k
$\tilde{S} = \{(u_1, w_1), \ldots, (u_{|\tilde{S}|}, w_{|\tilde{S}|})\}$
for $(u, w) \in \tilde{S}$ do:
  $b \leftarrow \mathsf{H}^{(l)}(u)$
  $\mathbf{z}_b \leftarrow \mathbf{z}_b \cup \{F_k(u\|w)\}$
  $\mathbf{z}_b \leftarrow \mathbf{z}_b \cup \{F_k(u\|\tilde{w}) \oplus 1 \mid \tilde{w} \in \tau_n(w)\}$

Online phase

Client		**C3S server**
Input: (u, w)		Input: $k, \mathbf{z}$
$r \leftarrow\$ \mathbb{Z}_q$		
$b \leftarrow \mathsf{H}^{(l)}(u)$		
$x \leftarrow \mathsf{H}_1(u\|w)^r$	$\xrightarrow{b,\, x}$	
		$y \leftarrow x^k$
$\tilde{z}_1 \leftarrow \mathsf{H}_2(u\|w, y^{\frac{1}{r}})$	$\xleftarrow{y,\, \mathbf{z}_b}$	
$\tilde{z}_2 \leftarrow \tilde{z}_1 \oplus 1$		
if $\tilde{z}_1 \in \mathbf{z}_b$ return match		
else if $\tilde{z}_2 \in \mathbf{z}_b$ return similar		
else return none		

Fig. 9. Algorithm for checking if a password similar to the user's password (w) is present in the leaked data ($\tilde{S}$). Here, $F_k(x) = \mathsf{H}_2(x, \mathsf{H}_1(x)^k)$, $\mathsf{H}^{(l)} : \{0,1\}^* \mapsto \{0,1\}^l$, $\mathsf{H}_1 : \{0,1\}^* \mapsto \mathbb{G}$ and $\mathsf{H}_2 : \{0,1\}^* \times \mathbb{G} \mapsto \{0,1\}^\ell$ are cryptographic hash functions.

VI. The MIGP Protocol

Our new protocol builds off first-generation C3 designs, specifically, the identity-based bucketization (IDB) mechanisms due to Li et al. [20] and Thomas et al. [26]. At a high level, the IDB protocol splits the leaked credential database into several buckets based on truncated hashes of usernames. The client and the service perform an OPRF-based private set membership protocol over the bucket to allow the client to check for equality. For MIGP, we propose to expand the leak data by storing the variations of the password for each bucket. Then the client can continue with the same protocol as IDB to learn if the queried password is same or similar to a leaked password associated with the same username. The complete protocol is given in Fig. 9.

Precomputation. In the existing C3 scheme, the server stores usernames and passwords in the form $F_k(u\|w)$, separated into buckets based on the l-bit prefix of the hash, $\mathsf{H}^{(l)}(u)$. Here k is a secret key chosen by the server. Here $F_k : \{0,1\}^* \mapsto \{0,1\}^\ell$ is a pseudo-random function, with key k. It is defined as follows, $F_k(m) = \mathsf{H}_2(m, \mathsf{H}_1(m)^k)$. H_1 and H_2 are cryptographic hash functions, where H_1 maps a binary string to an element in the group $\mathbb{G}$, and H_2 maps a binary string and a group element to a ℓ bit binary hash value. At least one of these hash functions should be slow and computationally expensive. For our implementation, we used H_2 to be slow (Argon2) and H_1 to be fast, as for our experiments in this section the choice has minimal impact. However, for rate limiting and abuse prevention, picking H_2 to be slow might be beneficial. We will discuss more on this choice in Section VII.

As we want the client to find out if the credential they query is similar to one stored by the server, we augment the leak data on the server in the following way. Let $\tau_n(w) = \{w_1, \ldots, w_n\}$ be n variations of a user u's leaked password w, after removing blocklisted passwords, if any. Then the server stores

$F_k(u\|w)$ corresponding to the leaked credential (u, w) and $F_k(u\|\tau_m(w)) \oplus 1$ corresponding to the password variations. The last bit of the PRF of similar passwords is "flipped" to differentiate it from the original leaked password.

These hashes are then separated into buckets based on the $\mathsf{H}^{(l)}(u)$ and duplicate hashes within buckets are removed. A duplicate may arise if the same variation $\tilde{w}$ is present in the similarity set of multiple leaked passwords. This could have a potential side channel attack on the bucket sizes especially due to such compression [18]. In practice, therefore, we recommend using more variations to ensure bucket size is exactly n times the number of leaked credentials.

Online computation. When a user u wants to check if their password w or a similar password is present in the MIGP server's database, they calculate the bucket ID of the bucket to query based on their username, $b = \mathsf{H}^{(l)}(u)$. They also pick a random value r and compute $x = \mathsf{H}_1(u\|w)^r$. Here, H_1 is a slow hash function that maps the arguments onto elliptic curve group element in $\mathbb{G}$. This is known as masking the user credentials, so that MIGP server does not learn anything about the queried credentials, except the bucket identifier b (l-bit of the hash of the username). The value of r is chosen randomly for each query. The server then raises the received x to its secret key k and send the resulting value y along with the bucket $\mathbf{z}_b$ containing blinded pairs computed by the server.

Once the client receives the response from the server, they can compute the PRF output $\tilde{z}_1$ after "unmasking" the server's response y using the random value r. They can then check to see if $\tilde{z}_1$ is present in the bucket, and if so learns that their queried password is present in the leaked data. If not, the client checks if $\tilde{z}_1 \oplus 1$ is present in $\mathbf{z}_b$, and if so it learns that the queried password is one of the (similar) variations of a leaked password that has been previously leaked along with their username. If none of the above conditions hold, it implies that the queried username-password is not present in the breached database.

A. Security Discussion

One of the key benefits of our protocol given in Fig. 9 is that it requires minimal changes to previously proposed [20] and currently deployed [26] protocols. This will make deployment simpler, and also the cryptographic security is derived directly from the underlying protocol. Here we briefly summarize the security achieved by MIGP, considering in turn malicious servers and malicious client threat models.

As in the prior protocols, the MIGP server learns only the client's queried bucket ID. This reveals some bits of information about the username, but nothing about the password. An actively malicious server can modify the result obtained by a client, e.g., by erroneously claiming passwords are not in the breach when in fact they are or vice versa. This is true also for deployed exact equality checking protocols [20, 26]. In theory one could try to use further techniques, such as having the server publish a commitment to the dataset and performing zero-knowledge proofs of (non-)membership [21]. We do not believe this is necessary for breach alerting as such input-switching attacks would seem to have low value to attackers.

An encrypted bucket reveals to a client the number of entries in the bucket, and updates to buckets will be revealed over time. We discuss this potential leakage further in Section VII. If the client has queried the OPRF on a matching username, password pair within the bucket, then it learns the encrypted metadata bit for that entry. Note that a malicious client can submit a query for a bucket for username u but submit an OPRF request for username $u' \neq u$. This is true as well for existing services. Thus rate limiting should not be based on bucket identifier, and instead on a client identifier (cookie or IP address) or a token mechanism such as Privacy Pass [16].

The way MIGP encrypts the bit indicating whether a match is a similar password or one found in the breach is malleable — a network adversary could flip these bits. But MIGP communications are protected with TLS, preventing such manipulations.

B. Performance Analysis

We implement the proposed MIGP protocol and perform experiments to estimate the computational overhead. First, we measure the precomputation overhead on the MIGP server-side for the two best-performing different similarity measures (see Section IV): (a) $n = 10$ variations based on Das-R and (b) $n = 100$ variations based on wEdit. The size of a bucket depends on the number of variations n and the length of the prefix l used for bucketing. To compare our implementation with existing C3 service, such as GPC [26] or IDB [20], we also implemented a variation of MIGP service with $n = 0$, which basically means that no variations of the leaked passwords are stored on the server.

For this part of the experiments, we consider 850 million username-password pairs from the compiled leaked data set [9] we collected. We experiment with two blocklist values $\beta = \{0, 10^3\}$. For $\beta = 10^3$, MIGP server blocks top 10^3 passwords as well as their n variations based on the setup.

We want to measure and compare the following three statistics for different setups of MIGP: (1) storage overhead on the server side, (2) required total bandwidth usage for running the protocol, and (3) latency of running the protocol.

Prototype implementation details. We prototype implement MIGP server and client in Python 3.8. We choose elliptic curve group secp256k1 for $\mathbb{G}$ and petlib[5] library to implement the required group operations. For hash H_1 we use slow Argon2 [7] with default parameter[6] (which takes about 152 ms seconds to compute on a Core i9 processor), and for H_2 we use SHA256. We set the security parameter $\ell = 128$, that means the output of the PRF computation is 128-bits or 16-bytes long. We build a web server application using Flask to instantiate MIGP server, and send queries to it using Python requests [7] library.

[5]https://github.com/gdanezis/petlib
[6]https://argon2-cffi.readthedocs.io/en/stable/
[7]https://requests.readthedocs.io/en/master/

l	w/o blocklisting avg.	w/o blocklisting std.	w/ blocklisting avg.	w/ blocklisting std.
16	1,297,460	23,340	1,245,410	22,310
20	81,090	5,820	77,840	5,560
24	5,070	1,450	4,860	1,390

Fig. 10. Average bucket size of MIGP with $n = 100$ variations for each password on the leak dataset, which contain 850 million username-password pairs. The average bucket size decreases exponentially as we increase the prefix length. By blocklisting top 10^3 password we can also decrease the average bucket size by 4.01% across all prefix lengths.

C3 service	n	l	B/w	Client side latency (ms) Crypto	Server call	Comp.	Total
IDB	0	16	0.9 MB	193	524	3	720
IDB	0	20	0.2 MB	194	373	3	570
MIGP	10^2	20	1.2 MB	197	702	12	911

Fig. 11. Average latency (in milliseconds) of computing MIGP with different number of variations n. Here $n = 0$ is a traditional C3 service such as IDB [20] with bucket prefix length $l = 16$. For MIGP with $n = 100$ we set the bucket prefix length to $l = 20$. Also shown the breakdown of time spent in different operations in the client. Here, "Crypto" include cryptographic operations including Argon2 hash and group multiplications, and "Comp." denote the time it takes to check if the computed OPRF value is present in the bucket or not.

For all the pre-processing and pre-computation of the data, we used a desktop with an Intel Core i9 processor and 128 GB RAM. Some of the computation (e.g., hash computations) can be expedited using GPUs, but we did not use any for our experiment. For running the server and client, we created two different AWS EC2 instances from the Ubuntu 20.04 LTS image located in two availability zones. The server is located in the Eastern region and the client on the Western region of the USA. The deployed server and the client are configured to be t2.medium and t2.micro, respectively.

Microbenchmark. For generating $n = 10$ variations for a password using Das-R similarity rules, it takes less than 0.01 ms, where as for generating $n = 100$ variations using wEdit similarity rules, it takes 0.61 ms. The time is average over 10,000 randomly chosen passwords from our dataset. Computing the Argon2 hash of a password takes 152 ms and a group exponentiation takes 0.77 ms. In total a PRF computation of a username-password pair takes 153 ms. A secp256k1 group element can be serialized as 33 byte string.

Precomputation overhead. MIGP (like prior C3 protocols) can precompute the PRF outputs of all the leaked passwords and their variations. Because MIGP requires storing n variations of the leaked passwords, it will require more time to precompute PRF values as well as will require more storage.

The precomputation on the server also involves computing the $F_k(x) = \mathsf{H}_2(x, \mathsf{H}_1(x)^k)$, where $x = (u\|w)$ for the breached password and $F_k(x) \oplus 1$, where $x = (u\|\tilde{w})$ for $\tilde{w} \in \tau_n(w)$. H_1 uses the slow Argon2 hash to slow down abuse of the protocol and protect the leaked passwords in the case C3 service database and the key is compromised. We hash the exponentiation values $\mathsf{H}_1(x)^k$ using H_2 and reduce their representation size to 16 bytes, which saves disk space and bandwidth.

As noted earlier that PRF computation takes 153 ms for each entry, computing PRF outputs for billions of username-password pairs and their variations will take a significant amount of computational cost. However, this computation is a one-time setup cost, and it can be easily parallelized.

The server has to store the precomputed values of the PRF output. As noted earlier each entry will cost 16-bytes, and the whole data will be split into buckets, as we describe below. We estimate the total cost of storing precomputed PRF outputs of 1 billion username-password entries and their variations would be 1.6 TB (considering every entry has 100 unique variations.)

Bucket size selection. To make the private set intersection protocols practical, C3 services use bucketing to partition the leak dataset based on the prefix of the hash of the username. MIGP follows the same approach. However, as MIGP contains n variations of each password, the buckets would be quite large for MIGP for the same number of buckets. This large bucket size will increase communication costs in terms of bandwidth and latency as the client has to download larger amount of data. We can reduce the bucket size by increasing the number of buckets — by increasing the length of the hash prefix l. The average bucket sizes for different hash prefix lengths for MIGP and $n = 10^2$ is shown in Fig. 10. The bucket size, as expected, decreases exponentially with the prefix length (l). Average bucket size for $l = 16$ (which is used by GPC [26]) is 1.3 million. This will require a bandwidth of 20 MB. The bucket size for $l = 20$ is 1.26 MB. The buckets cannot be compressed further as they are random hash values.

Blocklisting password reduces the effective leak data size. We found that blocklisting 10^3 most popular password reduces the database size by 4.01%. Buckets can be cached on the client side or served directly from CDNs (such as Cloudflare) as used by HIBP [6] to improve performance. Client side caching of buckets save fetching the same bucket again for checking different passwords for the same username. As most users have only a few email addresses, this can save significant network bandwidth.

Latency. We pick 25 random passwords from the test set and run the complete MIGP protocol for three settings $l \in \{16, 20\}$ for n=0 and l=20) for n=100 — the former settings is simulating existing C3 services such as GPC [26] or IDB [20]. The latter is simulating MIGP. We created random buckets for the simulation with size randomly sampled from the normal distribution with mean μ and standard deviation σ set to the mean and standard deviation we computed in Fig. 10, assuming with blocklisting 10^3 passwords.

The server and the client are executed in two different EC2 virtual machines located in two coasts of the United States, with average ping latency of 48 ms.

We report the average time taken for each protocol evaluation in Fig. 11. We also give the breakdown of the time spent in different operation for the client. To make a query to

MIGP, the client has to compute a slow hash of the username-password pair, mapping it to a group point and exponentiate. The over all time to prepare for a query takes 197 ms, majority of which is due to computing Argon2. This is a mechanism to slow down a malicious client from making to many guesses.

The server on the other hand has to do only do one group exponentiation using its private key k. The server sends back to the client the bucket of precomputed PRF values along with a group element. The total computational cost for the server is very small compared to the client.

As can be seen in Fig. 11, all computational cost of MIGP (with $n = 10^2$ and $l = 20$) and IDB (with $n = 0$ and $l = 16$) are very similar. The only overhead for MIGP is due to bandwidth and large bucket sizes. An average users will have to download the bucket only a few times (based on number of different email address they have). Therefore this cost will amortized over many calls to MIGP.

VII. Deployment discussion

MIGP service is an extension to existing C3 services, providing protection from credential tweaking attacks. MIGP stores a number of variations of the leaked password to achieve private similarity checking. The benefit of this approach is that it will be easier for currently deployed C3 services to move to MIGP. However, there are some deployment decisions that practitioners have to make before rolling out MIGP. Here we discuss the trade-offs that needs to be considered.

Rate limiting. As we show in Section V-C, an attacker can learn about the leaked credentials from MIGP servers quite effectively even for query budget $\leq 10^3$. Therefore, MIGP must deploy some form or rate limiting. Rate limiting based on the bucket id is ineffective. Rate limiting based on some API key or IP address can be done. Also, this means providing unrestricted access to the MIGP protocol would not be secure.

Removing bucket size side-channel. Bucket length depends on the leaked passwords of the users present in the bucket, especially for users with multiple passwords. To reduce side-channel due to bucket size, MIGP should ensure every user contain with m leaked password should have exactly $m \times n$ variants (after blocklisting and duplicate removal). Therefore the variation generation function should consider all the passwords of a user instead of only one. It is an open challenge (as also pointed out in [22]) how to take advantage of multiple leaked passwords of a user in credential tweaking attack or generating effective variations.

Bucket update. MIGP service (as other C3 services) have to periodically update their leaked data (as new leaks surface). This will require adding new credentials to the buckets. Updating the buckets with PRF outputs with the same key could be unsafe, as then an attacker can identify the newly added username-password pairs. Although it is unclear how to take advantage of this leakage, it will be better to avoid leaking such information. One approach is to rotate the OPRF key k every time there is a new leak. To do so, however, the server has to store the output of the group multiplication $\mathsf{H}_1(u\|w)^k$ on a safe location. Then the server can update the values to k' by raising old values to k'/k, and applying the fast hash to them. New credentials can be directly computed with $F_{k'}(\cdot)$.

Slow hashing before or after OPRF. As noted earlier MIGP uses two hash functions, and at least one of them must be slow. If H_1 is slow, then the output of H_1 can be stored on the server safely for key rotation. But an attacker can pre-compute these hashes offline and before launching an attack against MIGP (or a C3 service). The alternative approach is to make H_1 fast, and H_2 slow, and the benefit to that is that the client cannot do offline processing of the slow hash without communicating with the server, and if the server has strict rate limiting it will slow down the attack. GPC [26] implemented H_1 with the slower hash and H_2 with the faster.

How to use MIGP? MIGP can be used in place of existing C3 services to warn users during password entry process should their currently password or a variant of that appear in the breach data. Although we show that MIGP will be effective in flagging passwords vulnerable to credential tweaking attacks, the users have to take action, such as changing their password, to protect the account. As shown in prior work [24], despite repeated alerting only a small fraction of users change their password, MIGP will also face similar challenge. Especially the usability issues around communicating to the user that a similar password is leaked remains an open problem. Web services also can use MIGP as a signal to add additional monitoring to the accounts with password that are flagged by MIGP.

VIII. Conclusion

In this work, we tackled the problem of building an updated version of C3 systems, MIGP, that warns users from selecting passwords similar (and same) to a breached password in a secure manner. Via comparing different similarity metrics we show that computing variants of the password using weighted edit distance rules provide best combination of performance and efficacy. We give a secure protocol for MIGP. Underlying MIGP is a secure private similarity checking protocol (PSP). Despite secure PSP, MIGP can still be vulnerable to breach extraction attack, where an attacker can extract leaked (but not yet public) credentials from a C3 service, such as MIGP. We showed that an attacker can extract 14.33% of leaked passwords provide then can make at most a thousand queries per user. The attacker's success probability can be reduced significantly using blocklisting popular passwords. We developed a secure cryptographic protocol to perform PSI in the MIGP setting and implemented it to show that no additional computational overhead compared to C3 services. We finish with nuanced threat models and different discussions about deploying MIGP in practice.

References

[1] Argon2. https://www.npmjs.com/package/argon2/, 2018.

[2] Have I been pwned: API v2. https://haveibeenpwned.com/API/v2, 2018.
[3] I wanna go fast: Why searching through 500m pwned passwords is so quick. https://www.troyhunt.com/i-wanna-go-fast-why-searching-through-500m-pwned-passwords-is-so-quick/, 2018.
[4] Enzoic: Credential API. https://haveibeenpwned.com/Passwords/, 2020.
[5] John the Ripper password cracker. http://www.openwall.com/john/, Referenced March 2014.
[6] J. Ali. Optimising caching on pwned passwords (with workers). https://blog.cloudflare.com/optimising-caching-on-pwnedpasswords, 2018.
[7] A. Biryukov, D. Dinu, and D. Khovratovich. Argon and Argon2: password hashing scheme. Technical report, Technical report, 2015.
[8] J. Bonneau. The science of guessing: Analyzing an anonymized corpus of 70 million passwords. In *IEEE Symposium on Security and Privacy (SP)*, pages 538–552. IEEE, 2012.
[9] J. Casal. 1.4 billion clear text credentials discovered in a single database. https://medium.com/4iqdelvedeep/1-4-billion-clear-text-credentials-discovered-in-a-single-database-3131d0a1ae14, Dec, 2017.
[10] V. T. Chakaravarthy, V. Pandit, S. Roy, P. Awasthi, and M. Mohania. Decision trees for entity identification: Approximation algorithms and hardness results. In *Proceedings of the twenty-sixth ACM SIGMOD-SIGACT-SIGART symposium on Principles of database systems*, pages 53–62, 2007.
[11] R. Chatterjee, A. Athalye, D. Akhawe, A. Juels, and T. Ristenpart. password typos and how to correct them securely. *IEEE Symposium on Security and Privacy*, 2016.
[12] R. Chatterjee, J. Bonneau, A. Juels, and T. Ristenpart. Cracking-resistant password vaults using natural language encoders. In *Security and Privacy (SP), 2015 IEEE Symposium on*, pages 481–498. IEEE, 2015.
[13] R. Chatterjee, J. Woodage, Y. Pnueli, A. Chowdhury, and T. Ristenpart. The typtop system: Personalized typo-tolerant password checking. In *Proceedings of the 2017 ACM SIGSAC Conference on Computer and Communications Security*, pages 329–346. ACM, 2017.
[14] P. Crescenzi. A short guide to approximation preserving reductions. In *Proceedings of Computational Complexity. Twelfth Annual IEEE Conference*, pages 262–273. IEEE, 1997.
[15] A. Das, J. Bonneau, M. Caesar, N. Borisov, and X. Wang. The tangled web of password reuse. In *NDSS*, volume 14, pages 23–26, 2014.
[16] A. Davidson, I. Goldberg, N. Sullivan, G. Tankersley, and F. Valsorda. Privacy pass: Bypassing internet challenges anonymously. *Proceedings on Privacy Enhancing Technologies*, 2018(3):164–180, 2018.
[17] V. Enterprise. 2020 Databreach Investigation Report. https://enterprise.verizon.com/resources/reports/2020/2020-data-breach-investigations-report.pdf, 2020.
[18] J. Kelsey. Compression and information leakage of plaintext. In *International Workshop on Fast Software Encryption*, pages 263–276. Springer, 2002.
[19] H. Laurent and R. L. Rivest. Constructing optimal binary decision trees is np-complete. *Information processing letters*, 5(1):15–17, 1976.
[20] L. Li, B. Pal, J. Ali, N. Sullivan, R. Chatterjee, and T. Ristenpart. Protocols for checking compromised credentials. In *Proceedings of the 2019 ACM SIGSAC Conference on Computer and Communications Security*, pages 1387–1403, 2019.
[21] S. Micali, M. Rabin, and J. Kilian. Zero-knowledge sets. In *IEEE Symposium on Foundations of Computer Science*, pages 80–91. IEEE, 2003.
[22] B. Pal, T. Daniel, R. Chatterjee, and T. Ristenpart. Beyond credential stuffing: Password similarity using neural networks. *IEEE Symposium on Security and Privacy*, 2019.
[23] S. Pearman, J. Thomas, P. E. Naeini, H. Habib, L. Bauer, N. Christin, L. F. Cranor, S. Egelman, and A. Forget. Let's go in for a closer look: Observing passwords in their natural habitat. In *Proceedings of the 2017 ACM SIGSAC Conference on Computer and Communications Security*, pages 295–310. ACM, 2017.
[24] J. Pullman, K. Thomas, and E. Bursztein. Protect your accounts from data breaches with Password Checkup. https://security.googleblog.com/2019/02/protect-your-accounts-from-data.html, 2019.
[25] I. Sutskever, O. Vinyals, and Q. V. Le. Sequence to sequence learning with neural networks. In Z. Ghahramani, M. Welling, C. Cortes, N. D. Lawrence, and K. Q. Weinberger, editors, *Advances in Neural Information Processing Systems 27*, pages 3104–3112. Curran Associates, Inc., 2014.
[26] K. Thomas, J. Pullman, K. Yeo, A. Raghunathan, P. Gage Kelley, L. Invernizzi, B. Benko, T. Pietraszek, S. Patel, D. Boneh, and E. Bursztein. Protecting accounts from credential stuffing with password breach alerting. In *USENIX Security Symposium*. USENIX, 2019.
[27] Troy Hunt. Have I Been Pwned? https://haveibeenpwned.com/Passwords/, 2018.
[28] D. Wang, Z. Zhang, P. Wang, J. Yan, and X. Huang. Targeted online password guessing: An underestimated threat. In *Proceedings of the 2016 ACM SIGSAC conference on computer and communications security*, pages 1242–1254. ACM, 2016.
[29] Y. Zhang, F. Monrose, and M. K. Reiter. The security of modern password expiration: an algorithmic framework and empirical analysis. In *ACM Conference on Computer and Communications Security (ACM CCS)*, pages 176–186, 2010.

A. Top Das-R rules

These mangling rules are selected based on Das et al [15]. These are the top $n = 10$ variations based on our dataset:

- Capitalize or lower case the first character
- Delete the last character
- Delete second last character
- Delete third last character
- Insert '0' at the beginning
- Insert '1' at the end
- Insert 'a' at the beginning
- Insert 'q' at the beginning
- Insert '0' at the end
- Delete first character

B. Greedy approximation of OBDT and OMIGP

We can reduce a problem instance of finding optimal decision tree to determining the optimal tree for OMIGP. This is quite straight forward. We already show the reduction for problem instance of the OBDT to OMIGP in the paper. Therefore, these two problem reduces in an approximation-preserving manner [14], i.e. Any α approximation algorithm for optimal decision tree yields an α approximation algorithm for GreedyMIGP. Hence, the greedy algorithm GreedyMIGP also yields the same approximation bound as GreedyOBDT problem.

Claim 2. *In the algorithms* MIGPGuess*, we select the root node by selecting the password that have the maximum weighted ball. This condition is same the one used by GreedyMIGP (Fig. 6) for selecting root q^*.*

Proof: Let us consider, without loss of generality, that there are n passwords $w_1 \dots w_n$ with w_n password having the most weighted ball. We define $p(B(w_i))$ as the sum of probability of all the password in the ball corresponding to (w_i) i.e $p(B(w_n))$ is maximum among all passwords. We will show that we will select w_n as root node by the selection method in GreedyMIGP as well.

$$\begin{aligned} j^* &= \underset{w_i \in \{w_1 \dots w_n\}}{\operatorname{argmax}} \sum_{(x \in B(w))} p(x) \sum_{(y \notin B(w_i))} p(y) \\ &= \underset{w_i \in \{w_1 \dots w_n\}}{\operatorname{argmax}} p(B(w_i))(1 - p(B(w_i)) \end{aligned}$$

$z = p(B(w_i))(1 - p(B(w_i))$ is maximum when $p(B(w_i)$ = 0.5. The curve z is symmetric about 0.5, with the derivative being a straight line. Thus,

$$j^* = \underset{i}{\operatorname{argmin}} |p(B(w_i)) - 0.5|$$

Since $0 \leq p(B(w_i)) << 0.5 \forall i$, selecting the maximum weight ball will maximize the z. Therefore, GreedyODT algorithm is equivalent to selecting the most weighted ball.